

1 Workflow (Q&A)

1.1 Q: What preprocessing is required for KuaiRec before training?

A: You must run `dataset/kuaiREC/kuaiREC_process.py` to merge KuaiRec raw CSVs, build vocabularies for sparse/sequence features, create the feature schema (`description`), and write a preprocessed pickle used by the dataloader.

1.2 Q: What files does preprocessing produce that training depends on?

A: Preprocessing produces `kuaiREC_data.pkl` (train/test splits + schema) and also writes duration buckets and per-bucket play-time quantiles used by the D2Q baseline.

1.3 Q: What does the D2Q preprocessing artifact generation look like (schematically)?

A:

```
n_bins = 50
df["duration_bucket"], bins = pd.qcut(df["duration"], q=n_bins, labels=False,
                                     retbins=True, duplicates="drop")
desc.append(("duration_bucket", len(bins) - 1, "spr"))

quantile_num = 100
quantiles = np.linspace(0, 1, quantile_num + 1)
quantile_df = (df_train.groupby("duration_bucket")["play_time"]
               .quantile(quantiles).reset_index())
quantile_df.pivot(index="duration_bucket", columns="quantile",
                  values="play_time").to_csv(
    "d2q_duration_bucket_playtime_quantiles.csv"
)
```

1.4 Q: How do you run training/evaluation for a method?

A: Run the corresponding entrypoint (e.g. `python run_egmn.py --dataset_name kuaiREC`), which loads `./dataset/kuaiREC/kuaiREC_data.pkl` via `dataloader/kuaiREC.py`, trains for several epochs, evaluates on test, and prints MAE/XAUC/KL.

1.5 Q: What does the EGMN training loop structure look like (schematically)?

A:

```
for epoch in range(1, args.epoch + 1):
    for features, label in dataloaders["train"]:
        pi, lambda_, mu, sigma = model(features)
        nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                                   features["duration"].view(-1, 1))
        loss = nll + args.alpha * ent + args.beta * reg
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

1.6 Q: What dependencies does this repo assume?

A: The README indicates Python 3.x with PyTorch (tested around 2.1.0) and common scientific packages such as NumPy, pandas, and scikit-learn.